you ingest 10 GB of log data and 1 billion traces per day, with a retention period of a year. When you're troubleshooting, you don't want that fact to slow you down (wait for the queries to time out or take minutes to return), as this disrupts your flow, potentially causing you to overlook issues, like finding a bug slower—or not at all.

What is this environment where you ask these (observability) questions? These environments can be frontends and all-in-ones. Frontends are, by design, pure graphical user interfaces (GUIs) that allow you to configure one or more data sources to be used to create dashboards or alerts, such as Grafana. All-in-ones, on the other hand, combine backends and frontends into one indivisible logical unit. In other words, you can't use the GUI of an all-in-one as a standalone but only in combination with the backend(s) the provider designed it for.

So how can you pose different questions, and how are the results presented to you? You've guessed it: it is, indeed, frontend and all-in-one destinations you are using to achieve this. A term that you might come across in the observability community is *a single pane of glass*. In the context of this book, what I mean by this is a single place or application that you can use to consume all your signals, query, and correlate different signal types. Let's get into the deep end with a tour of observability frontends.

> **Tip**
>
> Reality shows that chasing the single pane of glass as the single most important requirement or priority (which may come from management) is often not the most efficient use of time and money. Can there be a standardization to consume many, if not most, of the signals in a single place? Yes. Will there be exceptions? Also yes. In other words: aim for it by considering one of the solutions presented in this chapter, but allow for flexibility where this is not (easily) possible.
>
> Note that I'm not suggesting that you should give up on the idea of standardizing on one main frontend, such as Grafana (it is very helpful to have all the data in one place, including the support of correlation). However, be prepared that there will always be a case where certain signals or sources are not covered by that frontend, requiring you to use a dedicated tool, in addition.

6.1 Frontends

We will first discuss the two most popular open source frontends: Grafana and Kibana. Next, we will have a look at other open source offerings and commercial offerings used in the observability space.

6.1.1 Grafana

Grafana (https://github.com/grafana/grafana) is nowadays part of Grafana Labs' LGTM stack (https://github.com/grafana)—Loki, Grafana, Tempo, and Mimir—its first and most important asset. Grafana started out as a fork of Kibana (http://mng.bz/x47W), when Grafana Labs' founder Torkel Ödegaard was looking for a UI that focuses on time series data and offers graph-focused dashboards.